Embedded Linux and Real-Time Operating System Development for Internet of Things Applications

Youssef Khaled Omar Mahmoud
Department of Electrical, Electronic and Communication Engineering
Faculty of Engineering, Cairo University
Giza, Egypt
youefkh05@gmail.com
ORCID: 0009-0005-5807-4381

***Abstract*—This paper presents the work completed during the IoT Summer Research Internship 2026 at the Nanoelectronics Integrated Systems Center (NISC), Nile University. The internship focused on embedded systems, Internet of Things (IoT), and real-time operating systems through a series of hands-on tasks. The developed solutions demonstrated communication between embedded devices and cloud services while providing practical experience in embedded software development, cloud technologies, wireless communication, and system integration.**

***Index Terms*—Internet of Things (IoT), Embedded Systems, Raspberry Pi, Embedded Linux, TI-RTOS, CC1352P1 LaunchPad, MQTT, HTTP, InfluxDB, Grafana, UART, Bluetooth, Zigbee.**

I. Introduction

The Nile University Summer Research Internship 2026 provided an opportunity to gain practical experience in embedded systems, Internet of Things (IoT), and real-time software development through a series of hands-on engineering tasks. The internship emphasized applying theoretical knowledge to real-world embedded applications while introducing modern software and communication technologies widely used in industry.

Throughout the internship, three major technical areas were explored. The first focused on Embedded Linux development using the Raspberry Pi, where Python was employed to interface with hardware peripherals and implement different GPIO control techniques. The second addressed the design of an end-to-end IoT system by transmitting sensor measurements from edge devices to cloud services using both HTTP and MQTT communication protocols. The collected data were stored in a time-series database and visualized through a real-time monitoring dashboard. Finally, the internship introduced real-time embedded software development using TI-RTOS on the Texas Instruments CC1352P1 LaunchPad, where multiple concurrent tasks including LED control, Bluetooth communication, Zigbee communication, and UART-based communication with the Raspberry Pi were implemented.

In addition to software implementation, the internship emphasized understanding the design trade-offs between different communication protocols and software architectures. The progression from basic hardware interfacing to cloud connectivity and finally to multitasking embedded systems provided a comprehensive view of modern IoT system development.

This report summarizes the assigned tasks, describes the methodologies followed during implementation, presents the achieved results, and discusses the technical knowledge and practical skills acquired throughout the internship.

II. Assigned Tasks

The internship activities were organized progressively over several weeks, with each stage building upon the knowledge and skills acquired in the previous one. [1]

Week 1: Embedded Linux and Raspberry Pi

- Raspberry Pi development using Embedded Linux.
- Python programming for hardware interfacing.
- GPIO programming and sensor interfacing.

Weeks 2–3: Internet of Things (IoT)

- Cloud communication using HTTP.
- Cloud communication using MQTT.
- Data storage using InfluxDB.
- Data visualization using Grafana.

Weeks 3–4: Real-Time Operating Systems (RTOS)

- Real-Time Operating System (RTOS) programming.
- Bluetooth communication.
- Zigbee communication.
- UART-over-USB communication.
- Integration between the RTOS microcontroller and the Raspberry Pi.

III. Brief Description of Assigned Tasks

The internship was organized into three major technical tasks that progressively introduced embedded systems, Internet of Things (IoT), and real-time operating systems (RTOS). Each task combined theoretical concepts with practical implementation, allowing the development of complete embedded applications and their integration into an end-to-end IoT system. [1]

A. Raspberry Pi Development

1) Objective: The objective of this task was to develop a solid foundation in software development on Embedded Linux while learning the fundamentals of hardware interfacing using the Raspberry Pi. The work progressed from C programming exercises to GPIO-based control applications, providing practical experience in software design, Linux development, and interaction with external hardware peripherals.

2) Task 2.1 – C Programming Fundamentals: The first stage focused on strengthening programming skills using the C language within the Raspberry Pi Linux environment. The implemented application solved the Emirp Number problem, where prime numbers whose digit-reversed values are also prime were identified within a user-defined range.

The program was developed using a modular design by separating the functionality into helper functions responsible for primality testing and digit reversal. Dynamic memory allocation was employed to store the resulting Emirp pairs, while structures were used to organize the associated data, including the prime number, its inverse, and the numerical difference between them. Input validation was also incorporated to detect invalid ranges and incorrect user entries before executing the algorithm.

This task provided practical experience with:

- Programming in C under Embedded Linux.
- Modular software development using functions.
- Structure-based data organization.
- Dynamic memory allocation using `malloc()`.
- Input validation and error handling.
- Compilation and execution using the GNU Compiler Collection (GCC).

The overall workflow of the developed Emirp application is illustrated in Figure 1. The program begins by validating the user inputs, allocates memory dynamically for storing valid Emirp pairs, iterates through the specified range to identify qualifying numbers, and finally computes statistical information before displaying the results. Example executions are presented in Figure 2, while representative input-output cases are summarized in Table I.

TABLE I: Representative execution scenarios of the Emirp number application. [1]

Input Range	Observed Result
11 – 100	Emirp pairs successfully identified and summarized.
101 – 500	Larger set of Emirp numbers processed correctly.
50 – 10	Invalid range detected and an appropriate error message displayed.
Negative or even inputs	Input validation rejected invalid values before execution.

Table I summarizes representative execution scenarios of the developed application.

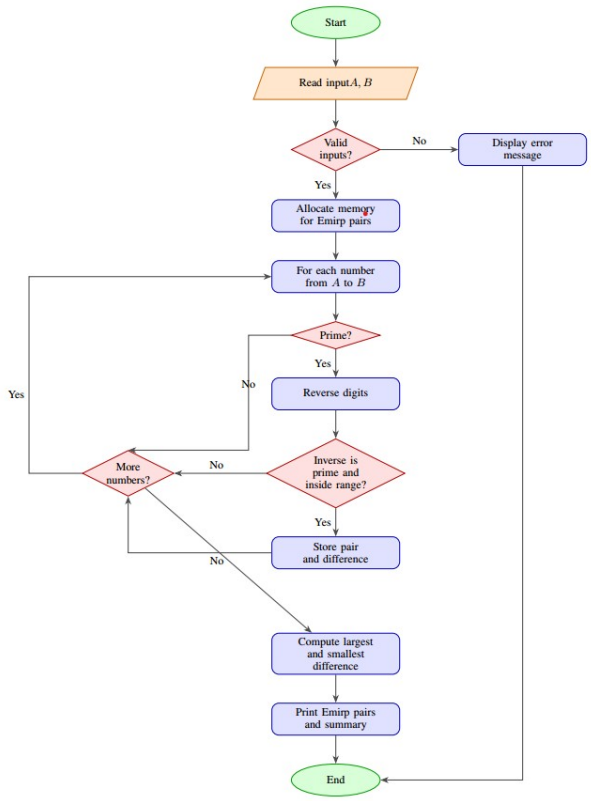
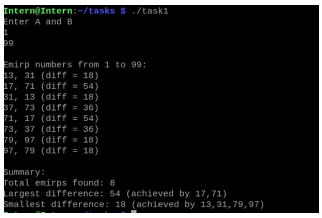
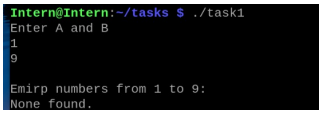


Fig. 1: Flowchart of the implemented Emirp number detection algorithm. [1]

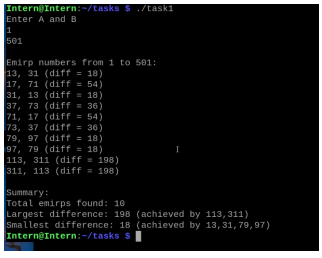
While Figure 2 presents sample terminal outputs obtained during testing.



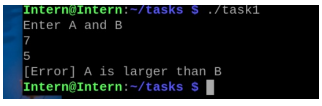
(a) Typical execution.



(b) Small range.



(c) Large range.



(d) Invalid input.

Fig. 2: Terminal outputs demonstrating successful execution and input validation of the Emirp number application. [1]

3) Task 2.2 – Embedded Linux and GPIO Programming: The second stage introduced hardware interfacing using the Raspberry Pi GPIO pins. Python was used to control LEDs

and read the state of a push button through the GPIO interface provided by the Linux operating system.

Three different software approaches were implemented for LED control:

- **Delay-based control**, where software delay loops generated the blinking period.
- **Timer-based control**, where timing mechanisms were used to improve execution accuracy.
- **Interrupt-based control**, where GPIO interrupts enabled immediate response to button presses without continuous polling.

These implementations demonstrated how different software architectures influence processor utilization, timing accuracy, and system responsiveness while using identical hardware.

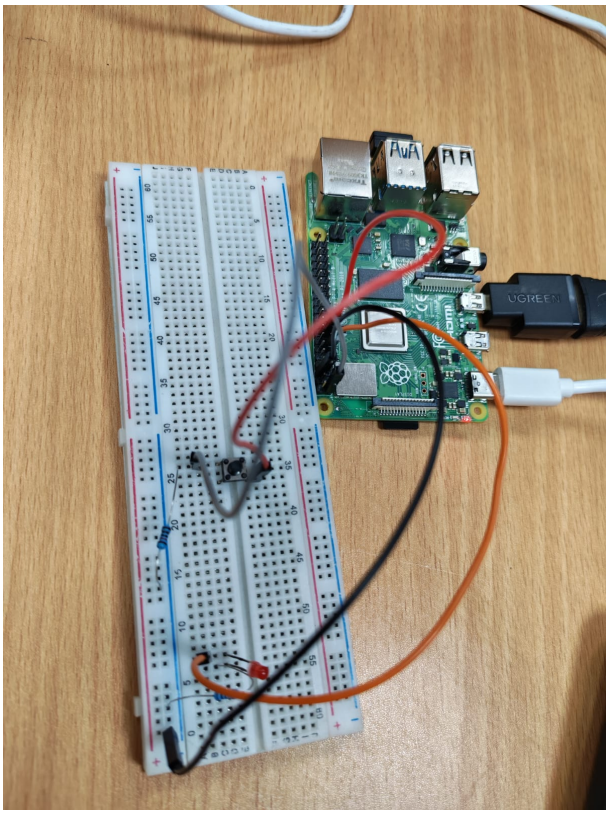


Fig. 3: GPIO-based LED control implemented on the Raspberry Pi. [2]

4) Outcomes: Example executions of the developed C application are presented in Figure 2, while Figure 3 illustrates the GPIO-based LED control implemented on the Raspberry Pi.

The Raspberry Pi tasks successfully demonstrated both software development and embedded hardware interfacing. The C programming exercise reinforced fundamental programming concepts, including modular design, dynamic memory management, and algorithm implementation. The GPIO applications provided practical experience in controlling external peripherals, implementing event-driven programming techniques, and comparing different software control methodologies. Figure 3 illustrates the implemented GPIO-based LED control setup.

Acknowledgment

The author would like to express sincere gratitude to the Nile University Research Office for offering the opportunity to participate in the 2026 IoT Summer Research Internship Program.

Special thanks are extended to Prof. Ahmed Soltan, Director of the Nanoelectronics Integrated Systems Center (NISC), for providing this valuable opportunity, his continuous support, and for taking the time to listen to and discuss our feedback throughout the internship.

The author also gratefully acknowledges the guidance and supervision of Eng. Salma, Eng. Omar, and Eng. Ahmed, whose technical support, constructive feedback, and mentorship were invaluable throughout the internship.

References

[1] Y. K. Omar, "NU-IoT-Research-Intern," GitHub repository, 2026. Available: https://github.com/youefkh05/NU-IoT-Research-Intern

[2] Y. K. Omar, "NU IoT Research Internship Task Demonstrations," Google Drive, 2026. Available: Google Drive Link